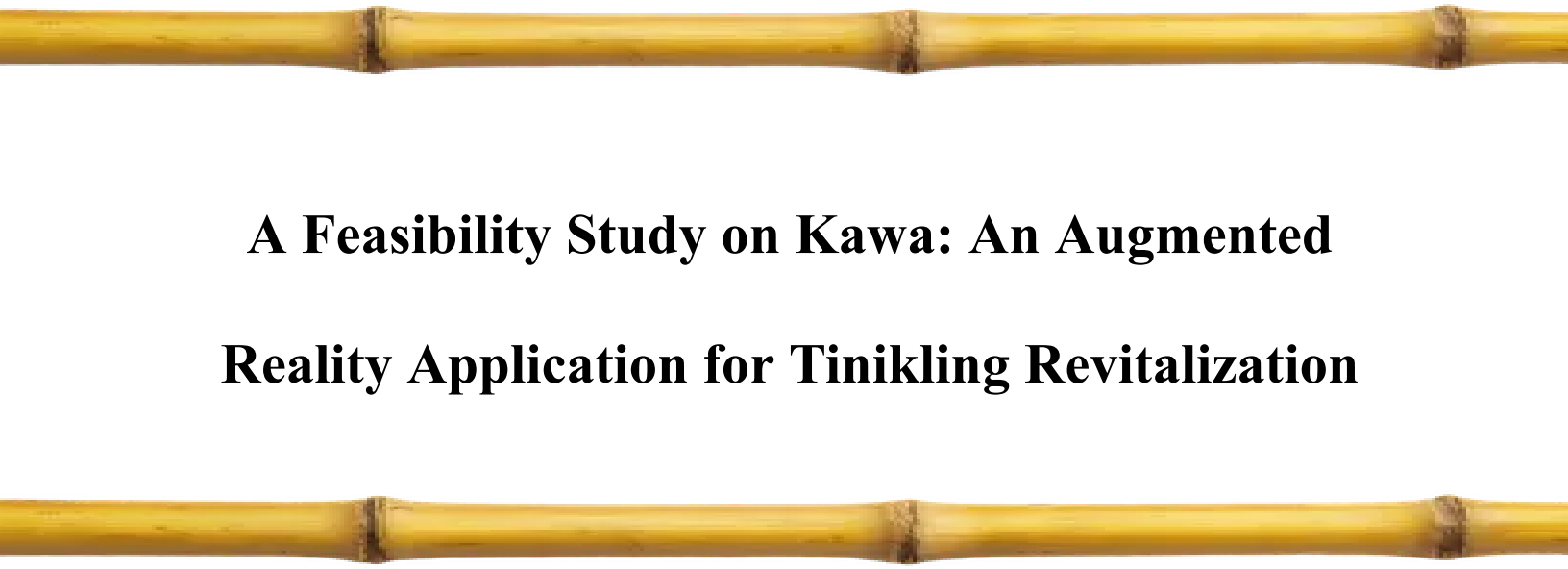


A Feasibility Study on Kawa: An Augmented Reality Application for Tinikling Revitalization

Justin Park

B.S. Candidate in Computer Science

Washington University in St. Louis

Table of Contents

i. Executive Summary....3

1. Introduction....5

2. Application Architecture and Design....7

2.1 The "Magic Mirror" Paradigm....7

2.2 Proposed Technology Stack....8

2.2.1 Frontend and Client-Side Architecture....8

2.2.2 Backend and Cloud Infrastructure....9

2.2.3 Augmented Reality and Computer Vision (AR/CV)....11

2.3 Application Workflow....12

3. Feasibility Analysis and Technical Mitigation....14

3.1 Latency and Processing Constraints....15

3.2 Visual Clutter and Environmental Variables....17

3.3 Pose Estimation Accuracy and Self-Occlusion....18

3.4 Collision Detection Mechanics....18

3.5 Haptic Feedback and Sensorial Immersion....19

4. Planting the Bamboo: Development Roadmap and Resources....20

5. References

List of Illustrations

1. **WashU Tinikling……. 6**
2. **Frontend Framework Decision Matrix……. 11**
3. **Backend and Cloud Infrastructure Decision Matrix……12**
4. **Augmented Reality and Computer Vision Decision Matrix…….14**
5. **Application Workflow and Processing Pipeline…… 15**
6. **Latency Budgeting Diagram……... 19**

i. Executive Summary

Tinikling, a traditional Filipino folk dance, faces a growing barrier to entry that threatens its cultural preservation. Organizing practices requires coordinating large groups of dancers alongside cumbersome physical equipment (long bamboo poles), often resulting in late-night practice times. For universities, which serve as primary hubs for vibrant culture in the US, this is detrimental. Late-night practices can negatively impact students' academic performance and endanger their physical safety. These logistical friction points reduce participation and make the art form increasingly inaccessible to new learners and returning dancers.

To address this accessibility crisis, this report proposes the development of a "magic mirror" Augmented Reality (AR) mobile application called *Kawa*, which is short for bamboo in Tagalog. By utilizing a device's camera to overlay virtual, rhythmically animated bamboo poles onto the user's physical floor, the application provides a solo, space-efficient practice environment. Specifically, the application will allow a single dancer to set the timing and positions of the virtual bamboo sticks and dance between them as they move according to their choreography. If the dancer “steps” on the virtual bamboo stick, they will receive haptic and audio feedback, and the move will be marked in a recorded replay of their dance. This encourages dancers to learn and practice the complex footwork of Tinikling anytime and anywhere, without the need for a full team or physical equipment.

Developing a rhythm-based AR application requires overcoming significant technical hurdles, primarily regarding latency and accurate motion tracking. This is because we must ensure that if a user “steps” on a virtual bamboo stick that they are notified in a timely manner such that they can react in time before the next part of the choreography begins. Our feasibility

analysis for *Kawa* concludes that modern mobile architecture can successfully mitigate these challenges:

- **Latency & Processing:** The application will utilize strictly on-device processing via Google's MediaPipe, a pose estimation model, decoupling the computer vision processing from the UI rendering. This eliminates network round-trip delays, ensuring the virtual bamboo remains perfectly synced with the audio track.
- **Pose Estimation Accuracy:** To handle the rapid, cross-stepping footwork of Tinikling without losing track of the user's overlapping feet, the system will utilize BlazePose's 33-landmark topology. This provides granular tracking down to the user's heels and toes.
- **Visual Clutter:** The application will programmatically prioritize high camera shutter speeds (60 FPS) to eliminate motion blur, paired with an environmental diagnostic tool to ensure users are practicing in adequately lit spaces.
- **Collision Detection & Haptics:** Detecting if a user steps on a virtual pole is achieved by mapping 2D camera coordinates into a 3D AR space. To replace the physical sensation of hitting a pole, the app will trigger immediate, low-latency haptic and audio feedback through the user's device, notifying them that they have made an error.

Ultimately, this feasibility study proves that preserving a century-old tradition does not require being anchored to the physical limitations of the past. The technical analysis confirms that modern mobile hardware and edge-computing models are fully equipped to handle the rigorous, high-speed demands of Tinikling. By transitioning large bamboo poles from the dance floor to the digital realm and completely eliminating the need to schedule practices, *Kawa* actively dismantles the steepest barriers to entry for Tinikling, transforming a localized, logistically challenging dance into a globally accessible, highly engaging consumer experience,

empowering the next generation to step into their heritage safely and independently. Proceeding with full development is strongly recommended.

1. Introduction

A cultural tradition is only as resilient as its ability to be practiced by the next generation. Today, the preservation of Tinikling is threatened not by a decline in cultural pride, but by severe logistical barriers to entry.



Figure 1: WashU Tinikling

Washington University in St. Louis students performing Tinikling at the 2026 Lunar New Year Festival. [Photograph by Ramses Baguio, 2026]

Tinikling is best described as a dance where people smile (Figure 1). It is a partner dance requiring a minimum of four people: two dancers and two people clapping and tapping the sticks. It is a complex, high-speed rhythmic performance that serves as a cornerstone of Filipino heritage. Its foundational footwork, where dancers step rapidly between clapping bamboo poles, is rooted in two historical narratives: local farmers imitating the evasive movements of the *tikling* bird dodging bamboo traps set out by rice farmers in an attempt to protect their crops, and a darker legacy of Filipino laborers evading the punishing bamboo strikes of Spanish plantation owners.

Formally preserved in 1946 by National Artist Francisca Reyes-Aquino in *Philippine Folk Dances*, the art form survives today not merely as a performance, but as a living celebration of resilience, rewriting the Philippines' dark colonial trauma into a beautiful act of defiance. Tinikling is not exclusive to the Philippines either. It has bloomed in the United States, especially in universities, as younger generations are putting their own inspirations and twists into the dance, transforming the dance entirely from what it was even just 100 years ago.

However, the modern preservation of this cultural cornerstone faces a severe logistical bottleneck. Based on my firsthand experience performing with the 16-person Tinikling act for the Lunar New Year Festival at Washington University in St. Louis, coordinating large group practices around demanding academic schedules is highly difficult. The strict requirement for large floor spaces and heavy, eight-foot bamboo poles frequently forced our group into grueling, late-night practice sessions. This rapidly impairs our cognitive abilities as students during daytime classes and introduces severe physical risks due to fatigue, which even led to a broken ankle for one of our members just one month before our performance date. In a dance where every single person, whether they are clapping sticks or dancing, is essential, these risks can turn into real problems that can jeopardize the entire performance and kill the act entirely. Nationwide, these immense spatial and scheduling constraints are significantly curbing the growth and modernization of Tinikling, deterring prospective dancers from engaging with their heritage.

To ensure the survival and continued evolution of Tinikling, a solution must be made for these physical barriers. This report proposes the development of a "magic mirror" Augmented Reality (AR) mobile application that overlays virtual, animated bamboo poles onto a user's floor, enabling safe, solo practice in any environment. For prospective investors and the

technical development team reviewing this proposal, the purpose of this report is to provide a decisive, evidence-based evaluation of whether this AR project is technically viable to fund and build.

This report evaluates the capabilities of modern mobile hardware, cross-platform software frameworks, and edge-computing computer vision models against the strict, high-speed demands of Tinikling. Furthermore, this report discusses the specific software architecture of the project, user-experience, and spends the most time investigating the technical feasibility of the application, identifying latency as the key limitation. This report will not cover post-launch marketing strategies, financial revenue modeling, or broader business operations.

To provide a comprehensive evaluation, the report will investigate two primary sections. First, it will outline the application's architecture and design, detailing the proposed technology stack and user workflow, followed by a justification for each decision. Second, it will present a rigorous feasibility analysis, breaking down the core technical challenges—including AR latency, visual clutter, pose estimation, and collision detection—and detailing the specific software strategies required to mitigate them before offering a final recommendation on development.

2. Application Architecture and Design

The revitalization of Tinikling through augmented reality (AR) necessitates a robust, low-latency software architecture capable of processing complex physical movements in real-time. Developing an application that successfully bridges the gap between physical folk dance and virtual training requires a meticulous selection of modern development frameworks, edge-computing computer vision models, and scalable cloud infrastructure. To evaluate this

system, the following section outlines the application's core visual paradigm, details the specific frontend and backend technology stack required for execution, and maps the chronological user workflow from initialization to haptic feedback.

2.1 The "Magic Mirror"

The proposed application operates on a "magic mirror". To utilize the application, the user places their mobile device or tablet on a tripod facing them, capturing their full body. The device's camera records the user and their physical environment, while the screen displays the mirrored live feed overlaid with virtual, moving bamboo poles. This setup allows the user to see themselves interacting with the digital bamboo in real-time, creating an accessible practice environment that completely circumvents the logistical hurdles of organizing a full dance team and sourcing physical equipment.

2.2 Proposed Technology Stack

To ensure cross-platform availability, low-latency processing, and rapid iteration, the application's technology stack is divided into three core domains: the client-side frontend, the cloud infrastructure, and the backend. The selection of these specific frameworks was determined through rigorous decision matrices evaluating performance trade-offs, scalability, and the ultimate goal of delivering a highly engaging, consumer-facing product. Each component was evaluated on a scale of 1 (Poor/Restrictive) to 3 (Excellent/Optimal) across key technical requirements.

2.2.1 Frontend and Client-Side Architecture

The application frontend must prioritize the fluid, user-centric experience required for modern consumer software while remaining accessible to the largest possible audience. React Native, utilizing Expo for deployment and hardware API bridging, was selected as the foundational framework.

While native development (SwiftUI for iOS or Kotlin for Android) offers maximum optimization for device-specific hardware, it fundamentally restricts the application's reach by requiring two entirely separate codebases. This violates the project's core mission of broad cultural accessibility. Conversely, React Native offers an optimal balance of cross-platform deployment and native-like performance. It was selected over alternatives like Flutter because the vast React/JavaScript ecosystem provides more mature and flexible integrations for complex augmented reality and camera hardware bridging compared to Flutter's Dart ecosystem.

Framework	Cross-Platform Reach	Native API Bridging	UI/UX Ecosystem	Total Score
React Native (Expo)	3	3	3	**9**
Flutter	3	2	2	**7**
Native (SwiftUI/Kotlin)	1	3	3	**7**

Table 1: Frontend Framework Decision Matrix

A comparative analysis of frontend frameworks based on cross-platform reach, native API integration, and ecosystem maturity.

As illustrated in Table 1, React Native achieved the highest cumulative score by offering the optimal balance of native bridging and cross-platform flexibility. Furthermore, Firebase will be seamlessly integrated into this client-side architecture to handle user authentication, profile management, and the real-time syncing of practice session telemetry.

2.2.2 Cloud Infrastructure

To support the client-side experience and manage intensive data loads, the backend infrastructure will be hosted on Amazon Web Services (AWS). This environment will utilize Docker containers to maintain consistent, isolated deployments for server-side rendering and asynchronous data processing tasks.

While a pure Backend-as-a-Service (BaaS) approach using only Firebase would accelerate initial development, it lacks the strict relational structure necessary for this application's core data. Persistent data regarding exact choreography timings, complex user progress metrics, and structured collision logs must remain highly structured and queryable. Therefore, a PostgreSQL database was selected over NoSQL alternatives (like MongoDB) because schema flexibility is an active detriment when dealing with strictly defined rhythm and motion data.

Infrastructure	Relational Structure	Environment Control	Complex Querying	Total Score
AWS (Docker) + PostgreSQL	3	3	3	**9**
NoSQL Stack (MongoDB)	2	2	2	**6**
Pure BaaS (Firebase Firestore)	1	1	1	**3**

Table 2: Backend and Cloud Infrastructure Decision Matrix
An evaluation of backend architectures emphasizing relational data structure, environment control, and complex querying capabilities.

As detailed in Table 2, AWS and PostgreSQL provides the necessary structural integrity for complex analytical queries that pure BaaS solutions lack. Additionally, Flyway will be

utilized to ensure reliable, automated database migrations and strict version control across all deployment stages.

2.2.3 Backend Augmented Reality and Computer Vision (AR/CV)

Because the application requires zero-latency feedback, the core AR and computer vision functionalities will avoid cloud processing entirely. Relying on remote servers (such as a Server-Side OpenPose architecture) introduces network latency that would instantly break the rhythm of the dance and render collision detection useless. Instead, the architecture relies on a hybrid, on-device edge-computing approach.

Native AR engines (ARKit for iOS and ARCore for Android) will be strictly responsible for environmental understanding, utilizing plane detection to accurately map the user's physical floor. Simultaneously, Google’s MediaPipe Pose will be overlaid to handle the lightweight, neural processing required to track the dancer. MediaPipe was selected because it provides high-fidelity, 33-landmark skeletal tracking while remaining heavily optimized for mobile Neural Processing Units (NPUs). While native body tracking (such as ARK it's built-in tracking) offers similar on-device latency, it restricts the feature solely to the iOS ecosystem.

AR/CV Solution	Low Latency (Edge Computing)	Platform Independence	Skeletal Granularity	Total Score
MediaPipe + Native AR Engines	3	3	3	**9**
Native Tracking (ARKit Only)	3	1	3	**7**
Server-Side OpenPose	1	3	3	**7**

Table 3: Augmented Reality and Computer Vision Decision Matrix
A technical comparison of AR and pose estimation solutions focusing on low latency, platform independence, and skeletal granularity.

Table 3 demonstrates this exactly, showing that MediaPipe combined with native AR engines guarantees that the system runs at the required high frame rates without compromising platform independence.

2.3 Application Workflow

To guarantee a seamless user experience with *Kawa*, the application workflow is divided into three distinct, sequential phases: Initialization, Execution, and Feedback (Figure 2).

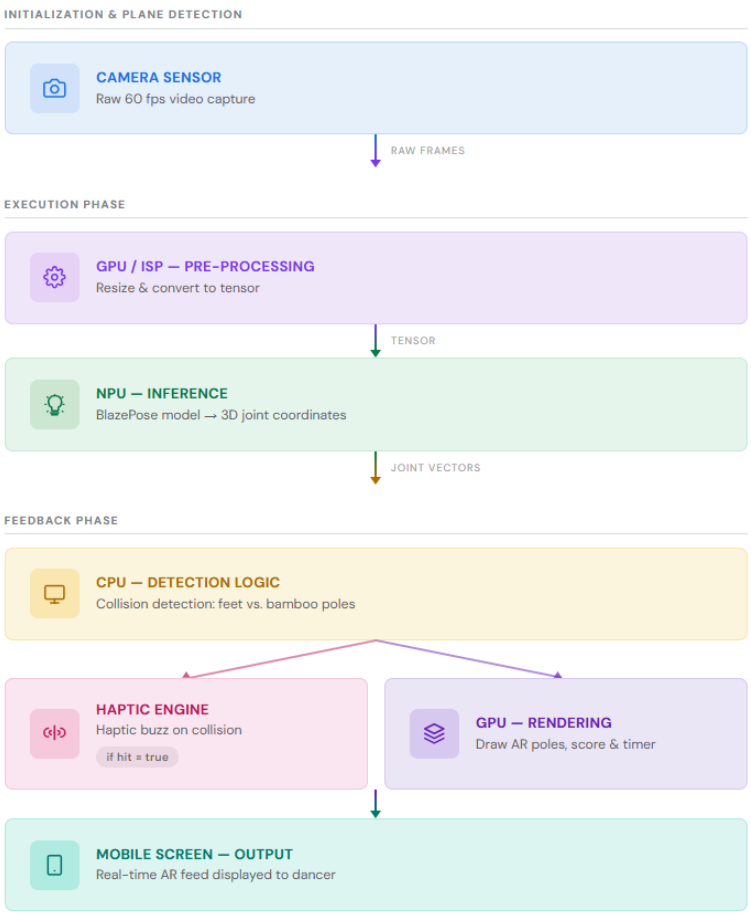


Figure 2: Application Workflow and Processing Pipeline

A chronological visualization of the application's on-device architecture.

- **Initialization and Plane Detection:** Upon launching a practice session, the application prompts the user to scan their immediate floor space. The native AR engine identifies the horizontal planes and anchors the virtual 3D models of the bamboo poles to the physical ground. The user then steps into the frame, triggering MediaPipe to establish a baseline pose skeleton and calibrate the dancer's height relative to the virtual objects.

- **Execution Phase:** The application begins playing the rhythmic Tinikling music. The virtual bamboo poles animate on the screen, sliding and clapping in strict synchronization with the music's beats per minute. The user performs the choreography,

stepping in and out of the virtual poles. During this phase, the device's camera continuously feeds frames to the user device's graphics processing unit (GPU).

To maintain a seamless, real-time experience at 60 frames per second (FPS), this entire computational pipeline must execute within a strict latency budget of 16.67 milliseconds per frame. Within this microscopic window, the GPU first transforms the raw digital camera feed into a tensor, which is a structured matrix representation of the pixels from the raw video. It then sends this tensor to the device's neural processing unit (NPU), which overlays the pose estimation model to track the precise spatial coordinates of the user's feet and ankles.

It is important to note however, that this 16.67-millisecond budget cannot be entirely consumed by computer vision analysis. We must also account for the time it takes to render all AR components on the user's screen. The GPU requires its own dedicated fraction of this time window to draw the 3D bamboo models, update the AR plane anchors, and refresh the user interface. Therefore, the application architecture explicitly decouples the rendering thread from the computer vision thread, ensuring that the NPU's skeletal tracking processes do not bottleneck the GPU's visual output, keeping the on-screen animation perfectly synchronized with the Tinikling music.

- **Feedback Phase:** As the user dances, the central processing unit (CPU) runs continuous intersection algorithms. If the user's foot coordinates occupy the same 3D spatial boundaries as the virtual bamboo pole during a "clap" frame, a collision is registered. The user receives immediate feedback, and the error is logged into the session's telemetry data for post-practice review.

3. Feasibility Analysis and Technical Mitigation

The ultimate success of this application hinges on its ability to accurately simulate the high-stakes, fast-paced nature of Tinikling. A dancer practicing in this virtual environment must implicitly trust the system's feedback; false positives (registering a hit when the user was safe) or false negatives (failing to register a real hit) will critically undermine the learning experience. To build this trust and deliver a viable consumer product, we must flawlessly execute across five interconnected technical pillars. First, because the dance is fundamentally rhythm-based, the architecture must overcome strict **latency and processing constraints** so the auditory beat and visual animations never desynchronize from the NPU's skeletal tracking.

Furthermore, the computer vision model must be robust enough to filter through **visual clutter and environmental variables**, such as messy living room backgrounds or severe motion blur from rapid jumping. Precise **pose estimation accuracy and handling self-occlusion** , which is when a dancer's legs rapidly cross and block the camera's view, is also a vital component for the video processing pipeline. Because the virtual bamboo lacks physical mass, the application must then rely on flawless **collision detection mechanics** to map these physical coordinates against 3D bounding boxes in real-time. Finally, to preserve the physical urgency that effectively trains a dancer's muscle memory, the application must replace the actual threat of the bamboo with immediate **haptic feedback and sensorial immersion**. The following analysis evaluates the feasibility of the system against each of these strict criteria, detailing the specific software strategies required to mitigate them.

3.1 Latency and Processing Constraints

In a rhythm-based dance like Tinikling, latency is the most critical point of failure. If the visual representation of the bamboo clamping shut is desynchronized from the audio beat, or if the collision detection algorithm takes too long to process a frame, the user experience collapses entirely. Previous studies have highlighted how network latency can severely degrade audio-visual perception during AR tasks, leading to user disorientation and poor performance (Hopkins et al., 2022).

To mitigate latency and ensure feasibility, the proposed architecture above strictly enforces on-device processing for all critical path operations. Offloading live video frames to a cloud server for computer vision analysis would introduce network latency (round-trip time) and packet loss risks, which are unacceptable for real-time dance feedback. While recent studies have explored utilizing wireless mesh networks and prefetching methods on edge servers to significantly reduce Web AR loading delays (Choi et al., 2023), transmitting continuous, high-capacity video data for real-time pose estimation still introduces network volatility that exceeds our strict frame budget. Building directly on this limitation, we make the decision to sever external network dependencies entirely during the execution phase, hard-coding the application to process the MediaPipe Pose model locally on the NPU. By utilizing MediaPipe Pose, which is heavily optimized for mobile NPUs, the application can achieve real-time tracking that exceeds 60 frames per second on modern hardware. Furthermore, the system will completely decouple the rendering thread (which draws the AR poles and updates the UI) from the computer vision thread. This multithreaded approach prevents the user interface from freezing or stuttering during complex pose calculations.

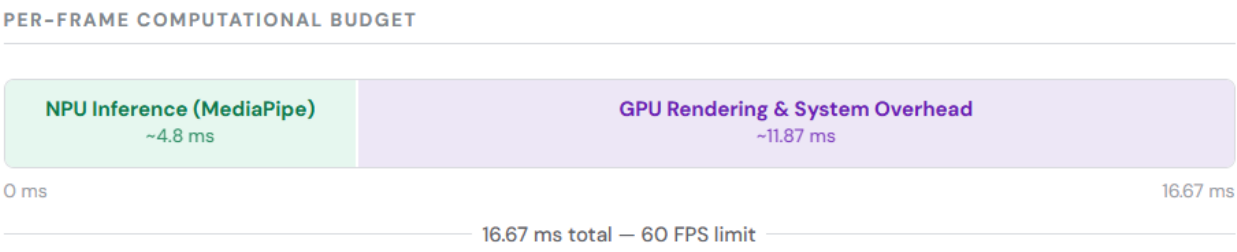


Figure 3: Latency Budgeting Diagram

Allocation of the strict 16.67-millisecond computational budget required to maintain a seamless 60 frames-per-second AR experience.

As shown in Figure 3, we have precisely 16.67 ms to analyze all video input and render all AR components. There are two ways of tackling this:

1. Render frames in batches, if latency becomes a significant issue during development, developers must accept that some frames will be dropped, or ignored, and only evaluate snapshots of the dance. This would mean analyzing every 4th or 5th frame of video at a time, leading to significantly more time to analyze video input.
2. Render every single frame. If latency is not a significant constraint to analysis, rendering every frame is possible. From preliminary research and understanding, the heaviest part of computation is the work that the NPU does when overlaying the pose estimation model onto the video. This is estimated to take around 4 to 5 milliseconds on average, which leaves around 11 to 12 milliseconds for other processes, which are trivial.

Interestingly, there is a parallel between the two most popular network communication protocols: Transmission Control Protocol (TCP) and User Datagram Protocol (UDP).

TCP focuses on reliability. It sends information one packet (a unit of network information) at a time, verifying that it sent the information, seeking verification from the receiver that it received the information, and verifying that the receiver actually got the

information. This ensures that no information is left unsent and this protocol persistently tries to send information if transmission fails.

UDP focuses on speed. It sends information out of order and as many packets as it can at a time, without needing any verification that the packets were received or sent. This protocol essentially just hopes that the receiver catches as many packets as it can. While it may seem haphazard, this protocol is widely accepted and very useful for real-time streaming services or gaming.

Rather than deferring this critical architectural decision to the development phase, this feasibility study definitively recommends adopting the UDP-parallel approach. In the context of a high-speed rhythmic dance, strict synchronization between the auditory beat and the visual bamboo animation is paramount. Guaranteeing the evaluation of every single frame (the TCP approach) introduces the fatal risk of the NPU bottlenecking the GPU, leading to visual stuttering and rhythmic desynchronization. Therefore, the system must prioritize computational velocity over perfect data retention. By strictly decoupling the threads, the application will be programmed to intentionally drop or skip CV frame evaluations when processing spikes occur. This ensures the 60 FPS rendering thread and audio track remain locked in perfect, uninterrupted rhythm.

3.2 Visual Clutter and Environmental Variables

Tinikling is characterized by rapid, hopping footwork. When capturing this movement via a standard mobile camera, motion blur becomes a significant obstacle, effectively turning the dancer's feet into a visual smudge. This visual clutter drastically degrades the accuracy of pose estimation models, which rely on clear edge detection and pixel contrast to map joints.

To address visual clutter, the application will programmatically override the device's default camera settings to prioritize a high shutter speed and a high frame rate (preferably 60 FPS). While this sacrifices some light intake for sharper frame capture, it is a necessary tradeoff for accurate tracking. Additionally, to optimize the AR system through rapid detection in cluttered environments (Bhakar & Bhatt, 2019), the app will include an environmental diagnostic tool during the Initialization Phase. This tool will analyze the camera feed and warn the user if the lighting is too dim or if the background is too noisy (e.g., heavily patterned rugs), ensuring the CV engine operates only when it has the cleanest possible visual data.

3.3 Pose Estimation Accuracy and Self-Occlusion

Standard pose estimation models frequently struggle with self-occlusion—for instance, when one leg crosses in front of the other, which is a foundational movement in Tinikling routines. Heavyweight, server-side models that can perfectly infer occluded joints are too computationally expensive for mobile devices, rendering them unfeasible for this use case.

The decision to use MediaPipe's BlazePose topology specifically addresses this feasibility concern. Unlike older models that only track a minimal skeletal structure, BlazePose utilizes an encoder-decoder network architecture optimized specifically for mobile devices, generating a highly granular 33-landmark topology (Bazarevsky et al., 2020). This level of detail is vital because the margin of error between a successful step and a bruised ankle in Tinikling is often a matter of inches. To operationalize this research, *Kawa*'s collision detection algorithm will be programmed to explicitly isolate and map only landmarks 27 through 32 (representing the left and right ankles, heels, and foot indices). By intentionally disregarding upper-body positional data during the intersection calculations, the system drastically reduces

computational load. Furthermore, MediaPipe employs temporal filtering across consecutive video frames. If a foot is temporarily blurred or occluded behind the other leg, the model utilizes the kinematic data and velocity from the previous frames to accurately predict its current location, maintaining tracking continuity even during the most complex cross-stepping choreography.

3.4 Collision Detection Mechanics

The core technical challenge of the application is accurately detecting collisions between a physical user and a virtual object. Because the user is not physically constrained by the virtual bamboo, the app must rely entirely on mathematically defined bounding boxes.

The application handles collision detection by continuously mapping the 2D coordinates extracted from the camera feed into the 3D coordinate space generated by the AR engine. The virtual bamboo poles are assigned 3D hitboxes that animate over time. Simultaneously, the pose estimation engine provides the spatial coordinates of the dancer's feet. The collision algorithm continuously checks for the intersection of the foot coordinates within the bamboo's active hitbox. To account for minor tracking inaccuracies and to prevent frustrating the user with overly punishing mechanics, the application will utilize a configurable tolerance buffer. The hitbox of the virtual bamboo will be dynamically scaled to be slightly smaller than its visual representation, providing the user with a grace margin that mimics the slight physical give of actual bamboo poles.

3.5 Haptic Feedback and Sensory Immersion

A physical Tinikling practice provides immediate, undeniable sensory feedback; when a mistake is made, the dancer physically feels the bamboo clamp on their ankle. To virtualize this experience and maintain the integrity of the practice session, the application must replace physical pain with distinct haptic and auditory cues. Research regarding motor control demonstrates that haptic guidance and vibrotactile stimulation significantly benefit motor learning in rhythmic and musical tasks, improving both physical timing and movement accuracy (Grindlay, 2008). Based on this finding, *Kawa* will not simply use haptics as a basic alert system, but as a core pedagogical tool.

Because the user is standing several feet away from the device on a tripod, relying on the mobile phone's internal vibration motor is not a feasible solution for haptic feedback. To solve this, the application architecture will support Bluetooth wearable integration. Users will be encouraged to wear standard smartwatches or fitness bands during practice. Upon a registered collision, the application will send a low-latency Bluetooth payload to trigger an immediate, sharp vibration on the user's wrist, coupled with a distinct, sharp "clack" sound effect emitted from the device speakers. This combination of audio-tactile feedback successfully trains the dancer's muscle memory to avoid the virtual poles with the same urgency as physical ones, fulfilling the final criteria for a truly immersive and effective AR revitalization tool.

4. Planting the Bamboo: Development Roadmap and Resources

Transitioning *Kawa* from a validated architectural concept to a deployed consumer application requires a phased, strategic development approach. The immediate next step is the creation of a Minimum Viable Product (MVP) focused exclusively on perfecting the core "magic mirror" AR overlay and the UDP-style pose estimation pipeline. This initial build will prioritize establishing

the strict 16.67-millisecond latency budget before integrating secondary features like user profiles or historical analytics. Following the MVP, rigorous alpha testing must be conducted with established cultural performance groups, such as WashU Tinikling dancers, to tune the collision tolerance buffers and ensure the digital hitboxes feel natural to experienced dancers.

To successfully execute this roadmap, specific technical and financial resources must be secured. The project requires a dedicated, multidisciplinary development team, specifically software engineers proficient in React Native and edge-computing computer vision integration, alongside a UI/UX designer to translate the complex AR mapping into an intuitive consumer interface. Financial allocation will be necessary not only for maintaining the AWS cloud infrastructure and PostgreSQL databases but also for acquiring a diverse array of mobile testing hardware. Securing a spectrum of low, mid, and flagship-tier iOS and Android devices, as well as various Bluetooth smartwatches, is essential to guarantee the haptic feedback and frame-dropping fail-safes operate universally, regardless of a user's economic access to high-end hardware.

By committing these targeted resources and initiating active prototyping, the theoretical framework of *Kawa* can be transformed into a tangible, high-impact tool. Surmounting these technical milestones will ensure that the vibrant legacy of Tinikling is not just preserved in theory, but actively propelled into the future, making the dance accessible and as resilient as the generations who created it.

References

Baguio, R. (2026). [Photograph of Washington University in St. Louis students performing Tinikling at the Lunar New Year Festival].

Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). BlazePose: On-device real-time body pose tracking. *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, Seattle, WA.

Bhakar, S., & Bhatt, D. P. (2019). Optimizing latency time of the AR system through glyph detection. *International Journal of Computers and Applications*, *41*(1), 14–31.

Choi, S., Hong, S., Kim, H., Lee, S., & Kwon, S. (2023). Prefetching method for low-latency Web AR in the WMN edge server. *Applied Sciences*, *13*(1), 133.

Grindlay, G. (2008). Haptic guidance benefits musical motor learning. *Haptics Symposium*, Columbia University.

Hopkins, T., et al. (2022). Studying the effects of network latency on audio-visual perception during an AR musical task. *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, Singapore, 26–34.